# Switching-Aware Precision Selection on Power-Capped GPUs: Mantissa Truncation versus INT8 Emulation

Koying Chen
Department of Computer Science, National Tsing Hua University, Hsinchu, Taiwan

***Abstract*—When a GPU runs at its power limit, the energy of each operation sets the clock. Zeroing low-order mantissa bits is known to lower GPU power in microbenchmarks; we ask whether doing it in software, on real data, is a usable optimization. On an NVIDIA GH200 cluster and two 8-GPU AMD MI350X nodes, the saving comes from bits that change between successive operands: the same values in sorted instead of shuffled order cost 8–9% less energy per operation, and the number of such bit flips explains 83% of the variance across layouts, whereas zero values and set bits do not. A first-order power model predicts truncated GEMM speedups across caps from one locked-clock measurement with a mean error of 1.5–2.1 percentage points (worst case 10.4). MI350X is power-bound at its stock limit, so in-program truncation speeds up FP64 HPL, HPL-MxP, a Gaussian-process likelihood and DF-MP2 by about 5–10% at default settings, with each code's accuracy check passed. Against INT8 emulation of FP64 the faster choice depends on the inner dimension, the cap and the tolerance; a per-call selector fitted on GEMM measurements alone picks it for 99% of the calls of a held-out DF-MP2 run and loses 0.02% against the per-call optimum. Of the two methods, only truncation applies to BF16 and FP8.**

***Index Terms*—power capping, energy efficiency, floating-point precision, mantissa truncation, GPU, mixed precision, HPL, emulation**

## I. Introduction

Power is becoming the ceiling on delivered performance. Data centers cap rack power, competitions cap cluster power, and accelerators ship with package limits that their own workloads reach: the AMD Instinct MI350X holds FP64, BF16 and FP8 matrix multiplication at its default 1000 W limit 82–100% of the time in our measurements. Under a limit the power manager lowers the clock until the device fits, so the speed of a program depends on how much energy each of its operations consumes.

The effect we build on is known. DGEMM on random operands draws 67% more power than on constant ones on A100 [1]. Gregersen et al. showed that zeroing the least significant bits of GEMM inputs lowers power, attributed it to fewer bit flips and proposed that compilers or schedulers exploit it [2]; a zero-input DGEMM on GH200 draws 563 instead of 898 W [3]; and an informal measurement showed that predictable GEMM data also runs faster under a lower power limit [4]. Hardware designs narrow multipliers or mask mantissas at unit inputs [5]–[8]. What these works leave open is whether the effect can be turned into a usable optimization: whether software truncation of *real* operands speeds up applications on production GPUs of different vendors while each application's accuracy check passes; whether the gain under a cap can be predicted rather than swept; and when it is the better choice than a format change such as INT8 emulation of FP64 [9], [10], whose speed also depends on the power limit. We answer these questions on two vendors' GPUs:

1) **Where the gain comes from and how much of it can be predicted.** Fills that stop the discarded bits toggling without making them zero retain 65–97% (ones) and 41–92% (fixed pattern) of the saving in matrix engines, FP64 vector units and memory copies of both vendors; with values and set bits held fixed, reordering operands alone changes the energy per operation by 8–9 points, and bit flips between neighbouring operands explain 83% of its variance (Section IV). If truncation only changes the switched capacitance, the truncated program under cap $P$ behaves like the native program under $P_0 + (P - P_0)/R$; with $P_0$ the measured idle power this first-order model predicts 42 cap–format GEMM points per platform with mean errors of 1.5 and 2.1 percentage points (at most 4.6 and 10.4), and two applications not used to build it with a mean error of 1.6 (at most 3.6; Section V).
2) **Gains that hold in real codes with their own accuracy checks.** On a GPU that is power-bound at its default limit (MI350X), in-program truncation speeds up HPL, HPL-MxP, DF-MP2 and a Gaussian-process likelihood by 4.6–10%, replicated on a second node; on GH200 the same codes gain under caps and save energy at the default. Safe bit counts must be validated per platform (Section VI).
3) **When to choose INT8 emulation instead, per call.** At equal accuracy emulation is faster for long inner dimensions at generous caps, truncation for short inner dimensions, tight caps and strict tolerances; limited memory and wide per-row exponent spans also favor truncation, and of the two only truncation applies to BF16 and FP8. A per-call selector fitted on GEMM measurements alone picks the faster method in 97.7% of held-out shape–cap–tolerance cases and for 99.3% of DF-MP2's calls and 8 of 9 GP configurations, neither used in its fit (Section VII).

Two measurement observations underpin these results rather than add to them: hardware energy accumulators avoid sampling bias, and chassis fans that react to the previous run change a power-capped GPU's speed by up to 4.5% through its temperature, which interleaving alone does not remove (Section III).

## II. Background and Related Work

**Input-dependent power.** Gregersen et al. varied value distribution, bit similarity and sparsity of GEMM inputs on A100, H100 and V100 and saw power differ by almost 40%, but avoided throttling [2]; He swept the A100 power limit [4] and Antepara et al. measured GH200 [3], [11], both with all-zero versus random inputs; Lucas and Juurlink modeled ALU power by the number of toggles [12]. None truncates real data, evaluates accuracy or applications, or compares vendors.

**Power-constrained HPC.** Manufacturing variation makes processors differ under the same power bound [13], [14]; overprovisioning [15] and power-limit tuning [16] decide how power is distributed. We lower the energy each operation needs, so the two compose.

**Masking, precision tuning and data movement.** Mantissa masking in hardware saves energy in multipliers [5], shaders [6], [17], [18], baseband processors [7] and register files [19]; DRAM and HBM energy depend on toggling [20]–[22]. Precision tuning [23], mixed-precision storage [24], [25] and transprecision computing [26] change types or bytes; zeroing changes neither. Low-precision LU with refinement underlies HPL-MxP [27], [28], and bit-information analysis can estimate needed bits from data [29].

**FP64 emulation and format changes.** The Ozaki scheme splits a product into exactly computed low-precision slices [30], [31]; Ozaki scheme II uses modular arithmetic [9] and is implemented in GEMMul8 for CUDA and HIP [10]; cuBLAS ships a fixed-point emulation with an automatic mode that picks the bit count from the exponent span [32], [33]. Tile mixed precision converts whole tiles of geostatistics factorizations to lower formats [34]–[38], and convergence-driven schemes switch formats or slice counts by solver state [39], [40]. All of these change the format; truncation keeps the original units, so the two can compose.

## III. Method and Experimental Setup

**Truncation.** We keep the top $k$ of the $p$ fraction bits of an operand (52 for FP64, 7 for BF16, 3 for FP8 E4M3) and zero the rest, by truncation (a bit mask) or round-to-nearest (add half a unit of the last kept bit first); sign and exponent are untouched. All bit counts in this paper are kept fraction bits; for HPL, whose parameter counts significand bits, 35 fraction bits correspond to its setting 36. HPL rounds to nearest even; applications round to nearest; GEMM microbenchmarks and HPL-MxP truncate. For uniform operands the relative error of $C = AB$ is $\approx \alpha 2^{-k}$ with $\alpha \approx 0.27$ (nearest) or 0.70 (truncation), independent of the inner dimension; truncation's error is a systematic shrinkage, which solvers with residual correction tolerate but accumulating processes do not. On real data $\alpha$ ranges from 0.07 to 9.6 with cancellation, so tolerable bit counts must be measured per application. The equivalent saving $\Delta W(P) = P_{\text{native}}(p'(P)) - P$ is the extra cap the native program needs to match the truncated one, interpolated along the native curve. For plane-wave DFT we tie the bit count to the Davidson threshold *ethr*: $k = \text{clamp}(\lceil -\log_2 ethr \rceil + m, 16, 52)$.

TABLE I
Where each code truncates, how often, the kept fraction bits (GH200 / MI350X; all codes round to nearest except HPL-MxP), and the extra work. All timings include it.

| Code | Operand, frequency | Bits | Extra work |
|---|---|---|---|
| HPL | $L$, $U$ panels, each update | 35 / 35 | pass over panels |
| HPL-MxP | BF16 storage, each write | 3 / 3 | fused |
| QE | Davidson blocks, each step | 16–42 / — | in place |
| DF-MP2 | tensor $B$, once | 20, 32 / same | one pass, $< 0.1\%$ |
| GP | panel $L_{21}$, each update | 28, 36 / same | fused |

**GH200.** Fourteen nodes with one GH200 480GB superchip each (Hopper GPU, maximum SM clock 1980 MHz, FP64 tensor peak 67 TFLOP/s [41]), 200 Gb/s InfiniBand, NVHPC 26.5 (CUDA 13.2). The module cap (GPU, CPU and memory) is set from 200 to 1000 W; the default is also the maximum.

**MI350X.** Two nodes with eight MI350X OAM GPUs each (gfx950, 288 GB HBM3E, maximum clock 2200 MHz), ROCm 7.2.3, Open MPI 5.0.7 with UCX. The per-GPU package limit is set from 1 to 1000 W; the default is the maximum; idle power is 244–275 W per GPU.

**Workloads.** GEMM microbenchmarks with cuBLAS or rocBLAS/hipBLASLt on uniform operands (FP64 $32768^2 \times 2048$; BF16 and FP8 $32768^2 \times k$ with $k = 6144$ or 8192). HPL and HPL-MxP: on GH200 8 nodes (HPL tuned, $N = 286720$; HPL-MxP $N = 491520$); on MI350X one node of 8 GPUs (HPL $N = 540672$, $1 \times 8$; HPL-MxP $N = 1034752$, 7.6 PFLOP/s natively); HPL-MxP uses a diagonally dominant test matrix, which its rules allow for testing only [42]. Quantum ESPRESSO 7.6 [43] (Si 1000 at 550 W, Au 112 at 450 W; GH200). DF-MP2 for taxol def2-TZVP (113 atoms): GPU4PySCF [44] on GH200; on MI350X, where GPU4PySCF does not run, the same three-index tensor built by PySCF on the CPU and contracted on the GPU (energy within $2\times10^{-13}$ $E_h$ of PySCF and $7.5\times10^{-8}$ $E_h$ of GPU4PySCF). An ExaGeoStat-like Gaussian-process likelihood on both ($n = 80000$, Matérn-3/2, blocked Cholesky with one FP64 DGEMM per panel). Truncation happens inside each program; GEMMul8 [10] and cuBLAS emulation replace FP64 GEMMs through a hook or the math mode. Table I lists where each code truncates.

**Measurement.** GH200: nvidia-smi samples every 250–500 ms, and energy integrates 1-s averaged module power. MI350X: a sampler reads all eight GPUs every 100 ms, including a 15.26-$\mu$J hardware energy counter (energy is its difference across a window) and the residency of power and thermal limiting. Every run records the cap in effect. Variants run in ABBA order on the same nodes or GPUs after a cool-

down; on MI350X all 8 GPUs run the same configuration at once.

**A pitfall: fans and temperature.** On MI350X the chassis fans react to the previous run: after a hot run the junction temperature falls by 25 °C within 20 s, and the next run can be up to 15 °C cooler and 4.5% faster, because cooler silicon leaks less. Interleaving alone does not remove this; it made 4 kept bits of HPL-MxP look faster than 3 at 800 W in one job. We therefore report two quantities: the raw paired ratio under the chassis' own fan control (what a user obtains), and the effect at equal temperature from a regression of log performance on variant and mean junction temperature ($-0.1$ to $-0.9$% per °C), which removes the confounder but, where truncation itself cools the GPU, also part of its benefit. Two controlled checks reproduce the adjusted values: at steady state (4 minutes of GEMM before each 2-minute window) the GEMM gains stay within 1.5 points of the short windows, and with a common 3-minute thermal load before every HPL-MxP run at 800 W the raw order is restored (3 bits $+10.4\pm2.8$, 4 bits $+6.0\pm3.5$%; adjusted 9.0 and 6.2%, as in the earlier jobs). Fan speed can be read but not fixed with our access.

**Statistics.** Each speedup is a paired ratio on the same node, cap and ABBA round. Measurements that share a chassis or a job are not independent, so on MI350X the unit is one measurement window of all 8 GPUs (GEMM), one node run (HPL, HPL-MxP) or one round of all 8 GPUs (GP, DF-MP2), and every MI350X experiment is repeated on the second node. On GH200 the units are node–pair samples from two node sets (HPL, HPL-MxP) or three nodes (GEMM), which treats pairs within a job as independent; those intervals may be too narrow. Even on MI350X, rounds and runs within one job on one node may remain correlated, so the intervals describe how stable a gain is on these two nodes, not its spread across a fleet of GPUs. A mixed-effects model (random intercept per ABBA block or round; node or GPU as fixed effects) gives the same estimates within 0.1 point and intervals of similar width ($\pm0.2$–$2.0$ against $\pm0.1$–$3.1$ points); with three or four blocks per node its variance components are poorly determined, so we report the $t$ intervals.

## IV. Why Fewer Mantissa Bits Save Power

Dynamic power is roughly $\alpha_{\mathrm{sw}} C V^2 f$ with a data-dependent switching activity $\alpha_{\mathrm{sw}}$. Under a cap the power manager lowers $f$ (and $V$) until the device fits, so a lower $\alpha_{\mathrm{sw}}$ becomes clock; below the cap it appears as lower power. Both occur on both platforms. On GH200 at its default 1000 W module cap the native FP64 GEMM is capped 95% of the time at 1822 MHz; with 0 kept bits it runs at the maximum 1980 MHz while module power falls from 984 to 707 W. On MI350X at its default 1000 W package limit the native FP64 GEMM is held at the limit 82% of the time at 1890 MHz; with 0 kept bits it draws 785 W at 2183 MHz and is 17% faster.

**Toggling or zero?** Zeroing stops the low bits from varying and makes them zero. Filling them with ones or a fixed nonzero pattern instead also stops them toggling but is not zero. On GH200 we compare module power at 1000 W or throughput at 500 W; on MI350X we lock the maximum graphics clock at 900, 1300 or 1700 MHz, below the frequency any configuration reaches, so that none is power-limited (PPT residency 0–0.6%) and the energy per operation compares switching activity directly (8 GPUs × ABBA).

TABLE II

Share of the zero-fill benefit (%) retained by a fill with ones / a fixed pattern. GH200: module power at 1000 W or throughput at 500 W; MI350X: energy per operation at locked clocks (GEMMs, range over three clocks), throughput at 600 W or power at 1000 W (other kernels). Last column: zero-fill benefit on MI350X (E/op: energy per operation).

| Kernel, kept bits | GH200 | MI350X | MI350X zero fill |
|---|---|---|---|
| FP64 GEMM, 0 | 76/68 | 73–75/69 | −20 to −29% E/op |
| FP64 GEMM, 20 (G), 27 (M) | 87/82 | 81–83/73–74 | −7 to −10% E/op |
| BF16 GEMM, 0 | 92/92 | 77–78/68–72 | −13 to −15% E/op |
| FP8 GEMM, 0 | — | 65–66/61 | −9 to −10% E/op |
| FP64 FMA loop, 0 | 87/62 | 74/63 | +33% speed |
| Memory copy, 0 | 97/86 | 83/41 | −9% power |

TABLE III

MI350X, FP64 GEMM at locked clocks: bit flips between neighbouring operands along $k$ and across rows of $A$ / columns of $B$, set bits per element, and energy per operation relative to native at 1300 / 1700 MHz (8 GPUs × ABBA; 95% intervals within $\pm0.5$ points). Sorted: the same values per row and column as shuffled, reordered along $k$. Krep / mrep: random dropped bits that repeat along $k$ / across rows.

| Bits | Layout | Flips $k$ | Flips row | Set bits | Energy/op (%) |
|---|---|---|---|---|---|
| 52 | shuffled | 27.2 | 27.2 | 34.5 | +0.6 / +0.7 |
| 52 | sorted | 23.2 | 21.2 | 34.5 | −7.3 / −8.6 |
| 27 | zeros | 15.2 | 15.2 | 22.8 | −8.4 / −9.6 |
| 27 | ones | 15.2 | 15.2 | 47.8 | −7.1 / −8.2 |
| 27 | krep | 15.2 | 27.7 | 35.3 | −0.6 / −0.5 |
| 27 | mrep | 27.6 | 15.2 | 35.3 | −3.7 / −4.2 |
| 0 | zeros | 1.7 | 1.7 | 9.3 | −24.5 / −28.6 |
| 0 | sorted | 0.0 | 0.0 | 9.3 | −32.0 / −38.1 |
| 0 | krep | 1.7 | 27.7 | 35.3 | −1.5 / −1.7 |
| 0 | mrep | 27.7 | 1.7 | 35.3 | −11.4 / −13.4 |

Most of the benefit survives such fills, in matrix engines, FP64 vector units and memory copies of both vendors (Table II), supporting the toggling hypothesis [2] beyond one vendor or unit. On MI350X the shares barely change with the clock (FP64, 27 bits: 81–83% at 900–1700 MHz), while the total saving grows with it (FP64, 0 bits: 20, 24 and 29% less energy per operation), as expected when $CV^2f$ grows relative to static power. The effect is not limited to matrix engines: an FP64 multiply-add loop with 0 kept bits draws 553 instead of 862 W on GH200 at 1000 W and runs 33% faster on MI350X at 600 W, and a memory copy, power-bound on MI350X at 600 W, runs 28% faster.

**Bit changes, not values.** Fills also change values. Three further controls on MI350X keep values or set bits fixed and vary only how bits change between the operands that meet in the matrix engine (Table III). Placing the same values per row and column in sorted instead of shuffled order along $k$ lowers the energy per operation by 8–9 points with 52 kept bits and by 8–9 points with 0, with identical values and set bits. Random dropped bits that differ between rows but repeat along

$k$ (krep) save almost nothing, whereas the transpose, random along $k$ but shared by all rows (mrep), saves about half as much as zeros: the engine sees operands from neighbouring rows as well as neighbouring $k$ in succession. Over the 15 layouts at each clock, the number of bit flips between neighbouring operands in both directions explains 83% of the variance of the energy change (0.45–0.53% per flip), set bits alone 44%, both together 89%. Bits that change between successive operands, not the value zero or the number of ones, are thus the main cause; the remainder fits partial products in which fixed ones still toggle against the other operand's kept bits.

**A second clock effect on MI350X.** With the clock locked and nothing power-limited, the effective clock still depends on the data: at a 1300 MHz lock the native FP64 GEMM runs steadily at 1171–1179 MHz (5th–95th percentile, 8 GPUs) and with 0 kept bits at 1235–1240 MHz; native BF16 at 1088–1100 and with 0 bits at 1132–1142 MHz, at 59–63 °C, while power and thermal residency and the per-XCD "clock below host limit" counters read zero. Circuit-level clock stretching or droop protection would explain it; we have no direct measurement. Truncation relieves it too.

## V. A Model of Truncation Gains Under a Power Cap

Let a GPU under cap $P$ settle at a target clock $f$. Its voltage $V(f)$ is set by the target clock, but switching happens at the effective clock $sf$, where $s \le 1$ is the data-dependent clock reduction of Section IV ($s = 1$ when there is none). With $c$ the program's effective switched capacitance and $P_0$ the power that does not scale with switching,

$$P = P_0 + c\,s\,V(f)^2 f = P_0 + c\,s\,g(f), \tag{1}$$

and performance is proportional to $sf$. Truncation changes $c$ and $s$, not $g$. For the native ($n$) and truncated ($r$) program define

$$R = \frac{c_r s_r}{c_n s_n}, \qquad \sigma = \frac{s_r}{s_n}; \tag{2}$$

at a common locked target clock, $R$ is the ratio of their dynamic powers and $\sigma$ the ratio of their effective clocks, both directly measurable. Under cap $P$ the truncated program settles where $c_r s_r g(f_r) = P - P_0$; the native program reaches the same target clock $f_r$ under $P'$ with $c_n s_n g(f_r) = P' - P_0$. Dividing,

$$P' = P_0 + \frac{P - P_0}{R}, \qquad p_r(P) = \sigma\, p_n(P'), \tag{3}$$

$$\Delta W_f = P' - P = (P - P_0)\left(\tfrac{1}{R} - 1\right), \tag{4}$$

where $p_r(P) = \sigma\, p_n(P')$ because both run at target clock $f_r$ and performance scales with $s$. $\Delta W_f$ is the extra cap the native program needs to reach the truncated program's *target clock*. The equivalent saving used in our results, the extra cap it needs to reach the truncated program's *throughput*, is

$$\Delta W = p_n^{-1}(\sigma\, p_n(P')) - P, \tag{5}$$

which equals $\Delta W_f$ only when $\sigma = 1$ (GH200, by assumption) and exceeds it when $\sigma > 1$ (MI350X, $\sigma = 1.01$–$1.05$). Every $\Delta W$ reported in this paper is measured, by interpolation along the measured native curve; the model is used only to predict

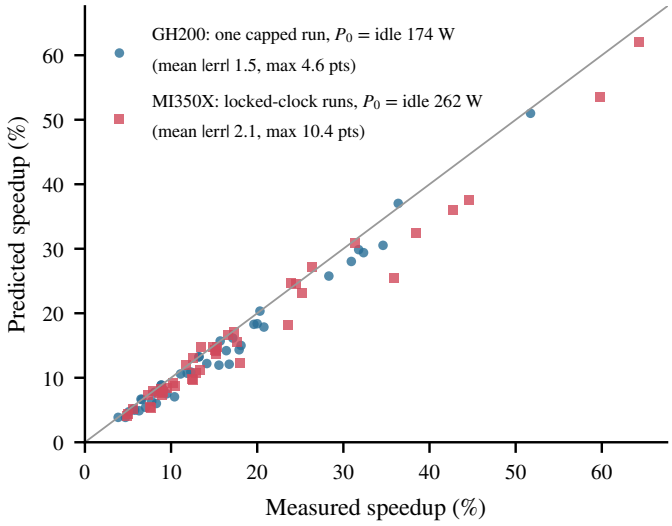



Fig. 1. Predicted against measured GEMM speedups under caps (FP64 with 0, 12/27 and 35/36 kept bits, BF16 with 0 and 3, FP8 with 0), with $P_0$ set to the measured idle power.

TABLE IV
Error of the first-order model in percentage points of speedup over 42 cap–format points per platform. Idle: $P_0$ = measured idle power; LOO: $P_0$ fitted on all other formats and applied to the held-out one; zero: all power treated as dynamic. Last column: largest error for FP64 with 27–36 kept bits.

| Platform | $P_0$ | Mean | Max | Max, FP64 27–36 |
|---|---|---|---|---|
| GH200 | idle 174 W | 1.5 | 4.6 | 2.9 |
| GH200 | LOO (190–200 W) | 1.0 | 3.6 | — |
| GH200 | 0 | 6.1 | 12.9 | — |
| MI350X | idle 262 W | 2.1 | 10.4 | 2.9 |
| MI350X | LOO (190–250 W) | 2.3 | 9.4 | — |
| MI350X | 0 | 4.0 | 17.2 | — |

speedups. The model assumes that $s$ does not change with $f$ over the range used and that the program stays compute-bound; predictions are capped at the program's clock-limited peak. The model needs the native curve $p_n(P)$ and $P_0$; it saves measuring every truncated variant under every cap. We set $P_0$ to the measured idle power, independent of the validation data: 174 W for a GH200 module (three nodes 169–179 W) and 262 W per MI350X GPU. The truncated kernel is characterized by locked-clock runs at 1300 MHz (MI350X; $R$, $\sigma$; no point of the cap sweep) or by its speed at the lowest cap (GH200, which we could not clock-lock; $\sigma = 1$; that point is excluded from evaluation), and predictions are compared with the GEMM sweeps at the other caps.

The mean absolute error is 1.5 percentage points on GH200 (largest 4.6) and 2.1 on MI350X (largest 10.4; Fig. 1, Table IV). Fitting $P_0$ on all other formats and predicting the held-out one gives 1.0 and 2.3; treating all power as dynamic ($P_0 = 0$) gives 6.1 and 4.0, and omitting $\sigma$ on MI350X 2.5. The ratios are $R = 0.94$, 0.89 and 0.67 for FP64 with 36, 27 and 0 kept bits, 0.92 and 0.85 for BF16 with 3 and 0, 0.90 for FP8 with 0, and $\sigma = 1.01$–$1.05$. For the FP64

widths applications keep (27–36 bits) the error is at most 2.9 points on both platforms; it is largest for BF16 with 0 bits at 400 W on MI350X, where the native GEMM barely runs, and for FP64 with 0 bits at 500–800 W, whose clock exceeds anything on the native curve. **Baselines.** Simpler predictors do worse or need more data. Applying the speedup measured at one cap to every other cap errs by 19 points on average on GH200 and 10 on MI350X (largest 73 and 48); predicting the speedup from the locked-clock energy ratio alone errs by 6.4 (MI350X), and the locked-clock clock ratio alone by 15.9. Linear interpolation between the speedups measured at the lowest and the highest cap errs by 6.0 on GH200 but only 1.8 on MI350X, against 1.7 and 2.0 for the model on the same points; it needs two capped runs of every truncated variant, whereas the model needs none. **External validation.** The GEMM points were used to develop the model. As a test on programs that played no part in it, we ran DF-MP2 and the Gaussian-process likelihood at the 1300 MHz lock on node n02, derived $R$ and $\sigma$ from their whole-program power and clock ($R$ = 0.92 and 0.86, $\sigma$ = 1.01 and 1.02 for DF-MP2 with 32 and 20 bits; $R$ = 0.96 and 0.93 for the likelihood with 36 and 28 bits), and predicted their speedups at 500–1000 W from their native curves on the same node. Over the 16 points the mean error is 1.6 percentage points and the largest 3.6. DF-MP2 is predicted within −1.6 to +2.4 points; the likelihood is underpredicted at every cap, by 0.6–3.6 points, perhaps because its Cholesky alternates GEMM-heavy and non-GEMM phases whose power differs, which one whole-program ratio does not capture. Once a platform's idle power and an application's native curve are known, a truncated program's curve thus follows from one locked-clock run to within a few points. The equal-clock part of the saving, $\Delta W_f \propto P - P_0$, also explains why the measured equivalent saving grows with the cap while the relative speedup does not: for FP64 with 36 bits on node n02 it rises from 12 to 72 W per GPU between 400 and 900 W, while the speedup falls from 8.9 to 4.9%. Errors of 1.5–2.1 points on average are small against gains of 5–20 points, so the model can estimate a gain and replace most of a cap sweep; they are not small against the one- or two-point margins near a crossover, and we do not claim that it picks the faster method there.

## VI. Results: Gains Under Power Caps

All results below are deployable: truncation happens inside the program, and the comparison is with the unmodified program. Fixed bit counts in Quantum ESPRESSO, GPU4PySCF and ExaGeoStat were also measured through a GH200 interposer that copies the operands; those numbers isolate truncation but pay a 3–9% copy cost, are slower than the unmodified program in two of three cases, and serve only as upper bounds (Supplement S1). Three third-party FP64 codes whose stored coefficients we truncated (MFEM, SW4, LAMMPS) gain 0–2%, because the truncated data are a small share of their traffic (Supplement S2).

TABLE V

HPL scaled residuals (threshold 16; repeated runs reproduce exactly). Kept fraction bits, rounded to nearest even. GH200: 8 nodes at 1000 W, $N$ = 286720 is the tuned configuration of the cap sweep; MI350X: one node of 8 GPUs at 1000 W. (F): fails the check; bold: the failures that rule out 27 bits.

| $N$ | Native | 35 bits | 31 bits | 29 bits | 27 bits | 23 bits |
|---|---|---|---|---|---|---|
| *GH200* | | | | | | |
| 131072 | $1.5\times10^{-4}$ | 0.16 | — | — | 13.97 | 456 (F) |
| 196608 | $1.3\times10^{-4}$ | 0.072 | — | — | **18.09 (F)** | 462 (F) |
| 262144 | $2.6\times10^{-5}$ | 0.042 | — | — | 14.70 | 91 (F) |
| 286720 | $7.6\times10^{-5}$ | 0.038 | — | — | 15.40 | — |
| *MI350X* | | | | | | |
| 262144 | $5.4\times10^{-5}$ | 0.46 | 11.1 | 84 (F) | **514 (F)** | — |
| 393216 | $4.6\times10^{-5}$ | 0.15 | 12.0 | 20.2 (F) | **104 (F)** | — |
| 540672 | $7.8\times10^{-6}$ | 0.27 | 3.5 | 17.3 (F) | **102 (F)** | — |

### A. GEMM

Zeroing speeds up GEMM at every cap on both platforms, though not monotonically as the cap tightens (Fig. 2). On GH200, FP64 with 27 bits is 8.1–11.1% faster at 500–800 W ($\Delta W$ 79–162 W) and BF16 with 0 bits 11.9–28.3%. On MI350X the default limit already binds: at 1000 W FP64 with 36, 27 and 0 kept bits is 5.1, 9.2 and 16.9% faster, BF16 with 3 and 0 bits 7.7 and 15.6%, and FP8 with 0 bits 8.8% (means over the 4 measurement windows of both nodes, 95% intervals ±0.6–1.6 points); at 500 W the same are 8.6, 15.2, 60.0, 13.5, 30.6 and 17.4% (±0.1–1.5). The two nodes agree within 1.8 points, and temperature adjustment changes no value by more than 1.1 points. The equivalent saving per GPU is 42–72 W for FP64 with 36 bits at 600–900 W.

### B. HPL and HPL-MxP

The residual check determines the usable bit count, must be checked at several sizes, and depends on the platform (Table V). On GH200 27 bits sits within one bit of the threshold; on MI350X it fails at every size, by 6–32×, and so does 29 bits. 35 bits passes on both with a margin of 35–100×; the platforms differ in HPL build and DGEMM blocking and accumulation order, so a bit count validated on one does not transfer.

**GH200, 8 nodes.** Tuned native HPL reaches 393.9 TFLOP/s at 1000 W. With 35 bits it is 3.3–4.7% faster at 500–800 W over 6 pairs from two node sets measured on different days ($\Delta W$ = 42–74 W per node at 500–700 W), and at the default cap mainly saves energy (7.5–8.6% per solve) with 0.6–3.9% more speed, depending on how far the cap and temperature pull the native clock down. The node sets differ by up to 0.9 points, more than pairs within a set, so node sets are the larger source of uncertainty (details in Supplement S6). HPL-MxP with 3 kept bits is 1.7–5.8% faster at 500–800 W; refinement grows from 1 to 3 iterations, which eats most of the LU's 5–16% speedup above 600 W; at 1000 W native HPL-MxP trips the chassis power brake at full fan speed, so we exclude that cap.

**MI350X, one node of 8 GPUs.** Native HPL reaches 424.9 and 429.8 TFLOP/s on the two nodes at 1000 W. With 35 bits it

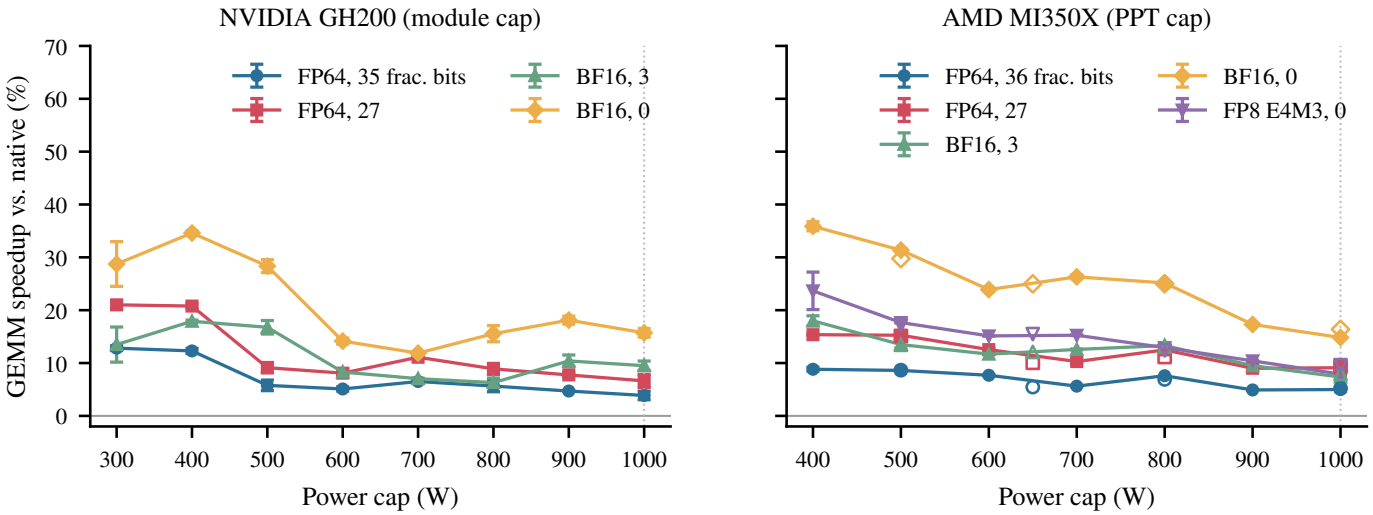



Fig. 2. GEMM speedup from mantissa zeroing under power caps. GH200 (module cap; FP64 $32768^2 \times 2048$, BF16 $k = 6144$): 95% $t$ intervals over 6 node–pair samples (3 nodes × 2 ABBA pairs). MI350X (package cap; BF16 and FP8 $k = 8192$): mean of node n02's two measurement windows, each averaging the 8 GPUs that run at once; bars span the two windows; open markers: node n01. The dotted line marks each platform's default limit, which is also its highest.

TABLE VI

MI350X: SPEEDUP (%) OVER THE UNMODIFIED PROGRAM AT THE DEFAULT LIMIT AND AT 500 W. POOLED: MEAN ± 95% $t$ INTERVAL OVER THE ANALYSIS UNITS OF BOTH NODES (WINDOW: ONE MEASUREMENT WINDOW WITH THE 8 GPUS AVERAGED; RUN: ONE NODE-WIDE RUN; ROUND: ONE ROUND OF 8 GPUS). PER NODE: RAW (SAME COOLING ENVIRONMENT) / ADJUSTED TO EQUAL JUNCTION TEMPERATURE.

| | | 1000 W | | | 500 W | | |
|---|---|---|---|---|---|---|---|
| Result | Unit, $n$ | Pooled | n01 raw/adj. | n02 raw/adj. | Pooled | n01 raw/adj. | n02 raw/adj. |
| GEMM FP64, 36 bits | window, 4 | 5.1 ± 0.6 | 5.2/5.4 | 5.0/5.5 | 8.6 ± 0.1 | 8.6/8.6 | 8.6/8.7 |
| GEMM BF16, 0 bits | window, 4 | 15.6 ± 1.4 | 16.4/16.6 | 14.8/15.2 | 30.6 ± 1.5 | 29.8/29.4 | 31.4/31.6 |
| GEMM FP8, 0 bits | window, 4 | 8.7 ± 1.6 | 9.6/9.5 | 7.9/7.9 | 17.4 ± 0.6 | 17.1/17.2 | 17.7/17.7 |
| HPL, 35 bits | run, 6 | 4.8 ± 0.3 | 4.6/4.5 | 5.1/4.8 | 7.0 ± 0.6 | 7.3/7.2 | 6.3/6.5 |
| HPL-MxP, 3 bits | run, 8 | 5.0 ± 0.4 | 4.9/4.8 | 5.1/5.1 | 19.3 ± 1.3 | 19.5/18.8 | 19.1/21.3 |
| GP likelihood, 28 bits | round, 4 | 7.4 ± 0.6 | 7.6/7.1 | 7.1/7.3 | 11.0 ± 1.0 | 10.8/12.5 | 11.2/12.9 |
| DF-MP2, 20 bits | round, 4 | 10.3 ± 0.6 | 10.2/13.3 | 10.4/14.2 | 18.6 ± 0.3 | 18.8/19.8 | 18.5/20.0 |
| DF-MP2, 32 bits | round, 4 | 5.7 ± 0.4 | 5.5/7.5 | 5.9/7.4 | 9.8 ± 0.1 | 9.7/10.5 | 9.8/11.0 |

is 4.6–5.1% faster at the default limit, 4.5–4.9% at 800 W, 6.2–8.0% at 650 W and 6.3–7.3% at 500 W (Table VI), with 4.6–6.8% less energy per solve and $\Delta W$ of 39–50 W per GPU at 650–800 W (about 0.35–0.4 kW per node). HPL-MxP, 7.61 and 7.65 PFLOP/s natively, reaches 7.98 and 8.04 PFLOP/s with 3 kept bits at the default limit (+4.9 and +5.1%) and gains 9.2–19.5% at 500–800 W, with 5.3–14.6% less energy per solve. Refinement grows from 2 to 4 steps (about 1.5% of the LU time). Keeping 0 bits does not converge on MI350X, so 3 bits is the operating point on both platforms. Apart from the 800 W cell discussed in Section III, temperature adjustment moves no HPL or HPL-MxP value by more than 0.7 points. For GP and DF-MP2, whose truncated variants run in the middle, hottest part of each round, it raises the gains by 0–4 points (DF-MP2 at 1000 W: 10.2 to 13.3% on n01); our controlled checks do not cover these codes, so we report the unadjusted values without claiming that they bound the effect. All HPL-MxP results hold for the diagonally dominant test matrix only: the code's generator always sets the diagonal to the row's absolute sum, and we have no implementation of a generator allowed for official runs, so neither the 3-bit operating point nor its gain is a general property of HPL-MxP; with exact factorization of the diagonal tiles 3 bits still passes (0.43) and is 4.4 and 9.4% faster at 1000 and 650 W. On two nodes (16 GPUs, $N = 1453056$) communication does not dilute the gain: 3 bits is 6.0% faster at 1000 W (14.61 to 15.48 PFLOP/s) and 16.5% at 500 W, but its residual rises to 11.8 against a threshold of 16 (0.43–0.5 on one node), so at larger $N$ 3 bits nears its limit.

### *C. Quantum ESPRESSO: Tolerance-Driven Bit Counts (GH200)*

Plane-wave DFT tolerates 20–28 kept bits; Gaussian-basis DFT needs 32–36 (Supplement S1). Tying the bit count to the Davidson threshold, in-program, reproduces the native total energy in every printed digit ($10^{-8}$ Ry) on both systems with the same SCF iteration count, while cutting time by 2.1–3.6% and energy by 2.2–3.4% against the unmodified program (Supplement, Table S6); $k$ rises from 16 to 39–42 as the SCF converges. On the gold surface about half of the gain comes from 7–9% fewer Davidson iterations, for reasons not

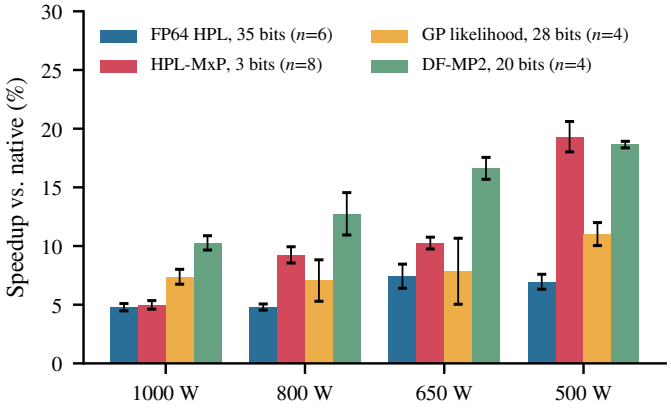



Fig. 3. MI350X: speedup of the in-program truncated variant over the unmodified program under each package cap, both nodes pooled. Bars: 95% $t$ intervals over analysis units, node-wide runs (HPL, HPL-MxP) or rounds of 8 GPUs (GP, DF-MP2), with $n$ in the legend; per-node and temperature-adjusted values in Table VI.

TABLE VII
DF-MP2 (TAXOL DEF2-TZVP): SPEED RELATIVE TO NATIVE (MEAN ± 95% $t$ INTERVAL; GH200: 6 ROUNDS FROM TWO JOBS ON TWO NODES; MI350X: 4 ROUNDS OF 8 GPUS, TWO PER NODE ON TWO NODES), AND THE ERROR $|\Delta E|$ OF THE MP2 CORRELATION ENERGY (TRUNCATED VARIANTS ROUND TO NEAREST). MI350X GEMMUL8 TIMES EXCLUDE THE ONE-TIME SETUP OF THE HOOK (SECTION VII-A).

| Cap | Native | 32 bits | 20 bits | Emulation |
|---|---|---|---|---|
| *GH200* (emulation: cuBLAS, 32 bits) | | | | |
| 1000 W | 39.5 s | +0.5 ± 0.2% | +0.8 ± 0.1% | −35% |
| 700 W | 43.9 s | +4.9 ± 0.4% | +7.4 ± 0.4% | −28% |
| 500 W | 51.5 s | +4.7 ± 0.2% | +7.7 ± 0.5% | −18% |
| *MI350X* (emulation: GEMMul8, 12 moduli) | | | | |
| 1000 W | 21.7 s | +5.7 ± 0.4% | +10.3 ± 0.6% | +66.6 ± 1.1% |
| 800 W | 26.4 s | +6.7 ± 1.4% | +12.7 ± 1.8% | +60.7 ± 0.9% |
| 650 W | 34.1 s | +9.5 ± 0.7% | +16.6 ± 0.9% | +44.8 ± 1.4% |
| 500 W | 55.8 s | +9.8 ± 0.1% | +18.6 ± 0.3% | +32.6 ± 3.1% |
| $\lvert\Delta E\rvert$ | | — | $\le 5\times10^{-12}$ | $\le 2.2\times10^{-8}$ | G: $2.2\times10^{-8}$; M: $7\times10^{-12}$ |

yet identified; on Si 1000 the work is unchanged, so its 2.1% is a power effect. The rule needs no safe bit count in advance and matches or beats the best fixed count.

### D. A Power-Hungry Application: DF-MP2

The three-index tensor $B$ is zeroed in place once; the contraction $B_i B_j^{\mathsf{T}}$ (FP64 DGEMM with the auxiliary basis as inner dimension, $k = 5477$) dominates the run. Keeping 32 bits changes the MP2 correlation energy by at most $5\times10^{-12}$ $E_h$ and 20 bits by at most $2.2\times10^{-8}$ $E_h$ (Table VII). On GH200 the truncated variants run at native speed at 1000 W with 8–14% less energy and gain 5–8% under caps; on MI350X, held at the limit 95% of the time at 1000 W, they are 5.7 and 10.3% faster there and 9.8 and 18.6% at 500 W (4 rounds on two nodes). Emulation behaves differently on the two platforms (Section VII-A).

### E. A Wide-Range Workload: Gaussian-Process Likelihood

The likelihood factors an $n\times n$ Matérn covariance by Cholesky; each panel $L_{21}$ keeps $k$ bits in one fused, coalesced pass before its trailing update. The panel rows span 162 binades, and 88% of their nonzeros lie more than $2^{-32}$ below their row maximum. With 28 bits the log-likelihood moves by a relative $3\times10^{-8}$–$4.5\times10^{-8}$, far below what matters for estimation, a one-parameter maximum-likelihood fit on GH200 returns the native estimate in every printed digit (Supplement S5), and a full three-parameter fit on MI350X (variance profiled, range and nugget by Nelder–Mead, two data sets) moves the estimates by at most 0.25% with 28 bits and 0.004% with 36, against a 2% difference in range between the two data sets. On GH200 the gain at 1000 W depends on how often native is capped (3.8% here, 0.1% with a Morton ordering that is capped less); on MI350X the limit binds and 28 bits is 7.4 ± 0.6% faster at 1000 W and 11.0 ± 1.0% at 500 W (4 rounds on two nodes). FP32 is unusable for estimation; truncating the FP64 tiles that remain in a tile mixed-precision Cholesky makes it a further 1.9–7.0% faster, so truncation composes with format changes (Supplement S5).

TABLE VIII
GAUSSIAN-PROCESS LIKELIHOOD ($n = 80000$, SIX RANGE PARAMETERS): LARGEST LOG-LIKELIHOOD DEVIATION FROM NATIVE AND SPEED RELATIVE TO NATIVE (MEAN ± 95% $t$ INTERVAL; GH200: 6 ROUNDS; MI350X: 4 ROUNDS OF 8 GPUS ON TWO NODES; TRUNCATION ROUNDS TO NEAREST). THE LOG-LIKELIHOODS ARE $1.2$–$1.7\times10^{5}$. [a] AT $n = 80000$ GEMMUL8 RETURNS SUCCESS BUT AN INVALID TRAILING UPDATE; AT $n = 64000$ IT IS 11–43% SLOWER THAN NATIVE (SECTION VII-A).

| | Trunc. 36 | Trunc. 28 | Emulation | FP32 |
|---|---|---|---|---|
| *GH200* (emulation: cuBLAS, 36 bits) | | | | |
| $\max\lvert\Delta\ell\rvert$ | $2.4\times10^{-5}$ | $7.2\times10^{-4}$ | $2.2\times10^{-3}$ | 87 |
| 1000 W | +3.0 ± 0.3% | +3.8 ± 0.4% | −41% | −8.2% |
| 700 W | +5.0 ± 0.1% | +7.9 ± 0.2% | −31% | +3.2% |
| 500 W | +4.8 ± 0.1% | +7.5 ± 0.1% | −21% | +3.9% |
| *MI350X* (emulation: GEMMul8) | | | | |
| $\max\lvert\Delta\ell\rvert$ | $1.9\times10^{-5}$ | $5.3\times10^{-3}$ | wrong[a] | — |
| 1000 W | +3.9 ± 0.2% | +7.4 ± 0.6% | — | — |
| 800 W | +4.1 ± 0.6% | +7.1 ± 1.8% | — | — |
| 650 W | +5.3 ± 1.7% | +7.8 ± 2.8% | — | — |
| 500 W | +6.9 ± 0.7% | +11.0 ± 1.0% | — | — |

TABLE IX
MI350X HEAD-TO-HEAD: SPEEDUP OVER NATIVE AT THE DEFAULT LIMIT / AT 500 W, WITH EACH METHOD'S ERROR. DF-MP2 AND GP: TRUNCATION TO 20 AND 28 BITS AGAINST GEMMUL8 WITH 12 MODULI, WHICH IS MORE ACCURATE (GEMM: FIG. 4). HPL: EACH METHOD'S FASTEST MEASURED CONFIGURATION WHOSE SCALED RESIDUAL STAYS BELOW HPL'S OWN THRESHOLD (16) OR, AS A SENSITIVITY CHECK, BELOW 1, AGAINST THE FASTEST NATIVE CONFIGURATION ON THE SAME NODE (TABLE X).

| Workload | Truncation | GEMMul8 |
|---|---|---|
| HPL, residual < 16 | +5.7/+7.3% (3.5/0.27) | +34/+12% (1.79) |
| HPL, residual < 1 | +5.1/+7.3% (0.27) | +17/−4.0% (0.033) |
| DF-MP2 ($\lvert\Delta E\rvert$, $E_h$) | +10/+19% ($2\times10^{-8}$) | +67/+34% ($7\times10^{-12}$) |
| GP, $n = 64000$ ($\lvert\Delta\ell\rvert$) | +5.9/+11.8% ($2\times10^{-3}$) | −11/−43% ($3\times10^{-7}$) |
| HPL-MxP (BF16) | +4.9/+19% (0.5) | no emulation |

## VII. THE BOUNDARY WITH INT8 EMULATION OF FP64

Table IX sets the two methods side by side at the best configuration each allows. Emulation wins by a wide margin where its fixed costs amortize at a generous cap (long inner

dimensions: HPL, DF-MP2, GEMM with $k \geq 2048$); a tighter cap shrinks its lead, and together with a tighter accuracy threshold can turn it into a loss; and it loses where the inner dimension is short (the Gaussian-process likelihood and GEMM with $k = 1024$). Truncation's gains are smaller but positive in every case we measured and larger at 500 W than at the default limit in every row; our in-program implementation needs no extra memory; and of the two strategies, only truncation applies to BF16 and FP8. The rest of this section explains the pattern.

### A. *Crossover: Shape, Cap and Memory*

Emulation trades FP64 arithmetic for several times as many integer operations, so its benefit grows with the ratio of INT8 to FP64 throughput, about 30 on GH200 and about twice that on MI350X. On GH200, cuBLAS emulation at the accuracy of 35-bit truncation ($\approx 10^{-11}$) is 11–28% slower than truncation on a square GEMM with $k = 2048$, and wins only for long inner dimensions; across 480 measurements a three-parameter model in $1/m + 1/n$ and $1/k$ describes GEMMul8 with $R^2 = 0.98$–$0.99$ (Supplement S4). On MI350X the crossover lies further toward emulation, and the cap moves it (Fig. 4). Truncation to 35 bits ($7.8\times10^{-12}$) is 5–6% faster than native at 1000 W and 8–9% at 500 W for every $k$ from 1024 to 8192, whereas GEMMul8 with 12 moduli ($\approx 5\times10^{-13}$) goes from −13 to +121% at 1000 W and from −45 to +48% at 500 W. Fig. 4 compares the two at equal accuracy, sweeping the kept bits (20–44) and the moduli (8–14). With $k = 4096$ emulation is faster at every accuracy except the tightest at 500 W. With $k = 1024$ the order depends on the accuracy and the cap: at 1000 W emulation wins for errors above about $10^{-10}$ (8 moduli: +27% at $1.3\times10^{-8}$, against +10% for 24-bit truncation) and truncation below (+4.8 against −6.2% near $5\times10^{-12}$); at 500 W truncation wins at every accuracy. Emulation draws the full limit, so under a cap its extra integer work is paid for with clock, whereas truncation removes work from the power budget; on GH200 the same holds for GEMMul8, while cuBLAS emulation, which draws only about 706 W at 1000 W, gains ground at lower caps: the direction depends on the emulation's own power.

**Applications.** In DF-MP2 on MI350X ($k = 5477$) GEMMul8 with 12 moduli is 34–67% faster than native and beats truncation at every cap, its lead over 20-bit truncation shrinking from 57 to 16 points between 1000 and 500 W; on GH200 cuBLAS emulation is 18–35% slower than native there (Table VII). In FP64 HPL on MI350X we fix a common threshold on the scaled residual and let each method use its fastest configuration that meets it, among those we measured (Table X). HPL's own check is 16. Tighter thresholds are a sensitivity analysis, not a requirement of HPL, and say nothing about what a given application needs (DF-MP2 and the Gaussian process speak to that). Under HPL's check, GEMMul8 with 10 moduli (residual 1.79; $N = 491520$, NB 4096, 4×2) leads at every cap: 576.8 TFLOP/s at 1000 W, 34% above the fastest native configuration on the same node, against 5.7% for 31-bit truncation, and 11.6 against 7.3% at 500 W. A

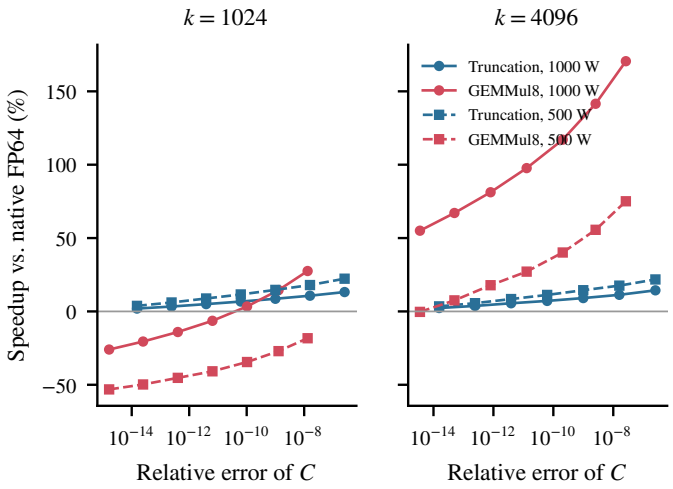



Fig. 4. MI350X, iso-accuracy comparison: speedup over native FP64 against the relative error of $C$ for truncation (round to nearest, 44 down to 20 kept bits) and GEMMul8 (8 to 14 moduli), $m = n = 32768$, at 1000 and 500 W (node n02, 8 GPUs × ABBA).

TABLE X
MI350X HPL, SENSITIVITY TO A COMMON RESIDUAL THRESHOLD: SPEEDUP (%) OF EACH METHOD'S FASTEST *MEASURED* CONFIGURATION WHOSE SCALED RESIDUAL IS BELOW THE THRESHOLD, OVER THE FASTEST NATIVE CONFIGURATION ON THE SAME NODE AND CAP.

| | Residual < 16 | | < 1 | | < 0.1 | |
|---|---|---|---|---|---|---|
| Cap | T | E10 | T | E12 | T | E12 |
| 1000 W | +5.7[a] | **+34** | +5.1 | **+17** | −0.1[b] | **+17** |
| 800 W | +5.0 | **+33** | +5.0 | **+15** | −2.3 | **+15** |
| 650 W | +8.0 | **+25** | +8.0[c] | +6.2[c] | +0.9 | **+6.2** |
| 500 W | +7.3 | **+12** | **+7.3** | −4.0 | **+5.6** | −4.0 |

16 is HPL's own check; 1 and 0.1 are a sensitivity analysis. Candidates (native and): 1 × 8, $N = 540672$ and 458752, 35 bits, all caps; 1 × 8, $N = 262144$ and 393216, 31, 35 and 39 bits, and $N = 540672$, 31 bits, 1000 W only; 4 × 2, NB 4096, $N = 491520$ (35 bits, 10 moduli) and 458752 (35 bits, 12 moduli), all caps. GEMMul8 does not fit at 1 × 8 or larger $N$. Not measured: 31 and 39 bits under caps, 39 bits at $N = 540672$. T: 35 bits ($N = 540672$, residual 0.27; for < 0.1: $N = 491520$, 0.033) unless marked; E: 10 moduli ($N = 491520$, 1.79) or 12 ($N = 458752$, 0.033). Bold: higher measured speedup. [a]31 bits (3.5). [b]39 bits at $N = 393216$ (0.039); truncation entries for < 0.1 are lower bounds. [c]No paired interval; order not resolved.

threshold of 1 excludes 10 moduli; 12 moduli (0.033) fits only up to $N = 458752$ and is 16.9% faster than the fastest native at 1000 W but 4.0% slower at 500 W, where 35-bit truncation gains 7.3%, a clear reversal. At 650 W truncation measures 1.8 points higher (8.0 against 6.2%), a difference between separate jobs for which we have no paired interval. At 0.1 truncation must also leave its best configuration, and among the measured configurations it leads only at 500 W. Tightening the threshold thus changes the measured order at the tightest cap.

In the Gaussian-process likelihood, whose trailing updates have $k = 1024$, GEMMul8 is slower than native at every cap ($n = 64000$, where it is correct: −11, −22, −37 and −43% at 1000, 800, 650 and 500 W with 12 moduli, log-likelihood within $3\times10^{-7}$), while 28-bit truncation gains 5.9–11.8%.

**Costs that appear only in applications.** Emulation needs workspace: at the $N = 540672$ of our best native HPL configuration the matrix fills 292 GB per GPU and GEMMul8's

TABLE XI

GH200, real data (1000 W): flop-weighted fraction of nonzeros more than $2^{-32}$ below their row maximum, and the total-energy error of each method. The "Nearest" rows keep 36 or 32 bits with round-to-nearest; cuBLAS emulation with 36 and 32 bits uses five slices, with 40 bits six; auto: its automatic mode (at most 79 bits). The Au 112 fraction comes from a two-iteration benchmark input, its errors from the fully converged input.

| | Valinomycin | QE Si 1000 | QE Au 112 |
|---|---|---|---|
| Below $2^{-32}$ | 24–44% | 0.24–0.54% | 0.008–0.02% |
| Nearest, 36 bits | $9.6\times10^{-7}$ $E_h$ | 0 Ry | 0 Ry |
| Nearest, 32 bits | $1.2\times10^{-5}$ $E_h$ | 0 Ry | $4\times10^{-8}$ Ry |
| cuBLAS emu., 36/32 | $\mathbf{5.25\times10^{-2}}$ $\mathbf{E_h}$ | $6\times10^{-8}$ Ry | $5\times10^{-8}$ Ry |
| cuBLAS emu., 40 | $8.4\times10^{-8}$ $E_h$ | — | — |
| cuBLAS emu., auto | $4\times10^{-9}$ $E_h$ | — | — |
| GEMMul8, 10 moduli | diverges | out of memory | $3.7\times10^{-7}$ Ry |

TABLE XII

Per-call selector on MI350X: share of cases in which it picks the measured fastest method that meets the tolerance, and the time lost against that choice. Leave-one-shape-out: each of 22 GEMM shapes predicted from the other 21 at 3 caps and 2 tolerances ($10^{-10}$, $10^{-12}$). DF-MP2: all 226 calls at 3 caps, measured per call (GEMMul8 with 12 moduli versus 20-bit truncation). GP: whole runs ($n$ = 64000, block size $k$; 12 moduli versus 35 bits). Neither application was used in the fit.

| Validation | Cases | Correct | Time lost |
|---|---|---|---|
| GEMM, leave one shape out | 132 | 97.7% | mean 0.09%, max 10.6% |
| DF-MP2 calls | 678 | 99.3% | 0.02% of run time |
| GP, $k$ = 1024–4096 | 9 | 8/9 | mean 0.3, max 3.1% |

workspace no longer fits; on a $1 \times 8$ grid it requests 100–114 GB even at $N = 458752$; with 12 moduli even $N = 491520$ fails; HPL must give up problem size before emulation can start. Through the BLAS hook, each new GEMM shape costs a large one-time setup: DF-MP2's first pass over its 226 shapes takes 138–141 s instead of 13 s, in GEMMul8 3.2.0 and in the current release 3.5.4 alike (the times above exclude this warm-up); calling the library directly with a workspace allocated once removes it (12.8–13.0 s in both passes), at the price of changing the application. And in the Gaussian-process likelihood at $n = 80000$, the hooked DGEMM of the first trailing update ($m = n = 78976$, $k = 1024$) returned success but an invalid result in every one of 256 runs, without a diagnostic. A minimal reproduction on uniform operands localizes it exactly: through the library's direct API and through the hook alike, $C = AB^{\mathsf{T}}$ with $k = 1024$ is correct ($1.6\times10^{-15}$) for $m = n = 65535$ and wrong (relative error 31, all entries finite) from $m = n = 65536$, i.e., once $mn$ reaches $2^{32}$, consistent with a 32-bit size overflow; it persists in the current release (3.5.4). It is an implementation defect, independent of the data, not a property of emulation; every other emulated result in this paper stays below this size, and we report it only as a deployment risk of replacing a BLAS call wholesale.

### B. Exponent Span: When Emulation Silently Fails

Truncation keeps every element's exponent; Ozaki-type emulation slices each row relative to its maximum, so elements far below the maximum lose bits. On data with a wide per-row span this can go wrong silently (Table XI): in Gaussian-basis DFT on GH200, where 24–44% of the flop-weighted nonzeros lie more than $2^{-32}$ below their row maximum, fixed 36-bit cuBLAS emulation lets the SCF converge normally to a total energy 52.5 m$E_h$ off, and GEMMul8 diverges, whereas rounding to 36 bits errs by $9.6\times10^{-7}$ $E_h$. With 40 bits or the automatic mode the emulation is correct but 1.5–2.1× slower than truncation or native. Uniform synthetic spreads understate the problem: on both platforms, spreading element exponents uniformly over up to 64 (MI350X) or 256 (GH200) binades barely affects GEMMul8 (MI350X, 12 moduli: median error $5.7\times10^{-13}$–$8.0\times10^{-13}$, p99 at most $4.4\times10^{-11}$). A wide per-row span, which an interposer can measure cheaply beforehand (Supplement, Fig. S1), is therefore a necessary but not sufficient condition for this kind of failure, the precision loss of a fixed-width emulation; narrow-span codes such as HPL, Quantum ESPRESSO and DF-MP2 do not suffer it. It does not cover implementation defects such as the size overflow of Section VII-A, which occurs on narrow-range uniform operands as well. Replacing every FP64 GEMM with FP32 or 3xTF32 fails for a different reason, accumulation over long inner products (Quantum ESPRESSO, Supplement S3).

**Truncation inside emulation.** Truncating the operands before GEMMul8 might lower the INT8 engines' toggling. On MI350X it adds 1–4 points under a cap (12 moduli: 18.4 to 22.6% over native at 500 W with 19-bit operands) and nothing at 1000 W, while the error becomes that of the truncation ($5.1\times10^{-7}$); 8 moduli on full operands are faster and more accurate. Within emulation, the number of moduli is the knob to turn.

### C. Choosing per Call

The pattern above can be turned into a per-call rule. For a GEMM call $(m, n, k)$ under cap $P$ with a tolerated relative error $\varepsilon$ of the product, each method takes its cheapest setting whose measured error is at most $\varepsilon$ (kept bits, or moduli), and the method with the smallest predicted time wins. Truncation's speedup barely depends on the shape (35 bits: 4.6–6.8, 5.0–7.4 and 8.0–9.8% over 22 shapes at 1000, 650 and 500 W), so it is one number per cap, or the model of Section V. Emulation's time relative to native is modelled as $r_e = a + b/k + c\,(1/m + 1/n)$ per cap and number of moduli, fitted on 22 shapes ($m = n$ from 4096 to 32768, $k$ from 512 to 8192, two rectangular; GEMMul8 3.5.4, $R^2$ = 0.97–0.99); calls through the hook add its one-time setup per shape. The rule uses only GEMM measurements.

The rule picks the fastest method in 97.7% of held-out shape–cap–tolerance cases (Table XII); of the three misses, two cost 1.3% or less and one 10.6% (a $k = 4096$ shape at 500 W near the crossover). On DF-MP2, which played no part in the fit, the predicted emulation-to-native time ratio of each call is within 0.05 (1000 W) to 0.13 (650 and 500 W) of the measured one, and the choice is right for 99.3% of the 678 calls and 99.9% of their time, so a run that follows

it loses 0.02% against the per-call optimum. With $k = 5477$ that choice is emulation for every large call. In the Gaussian-process likelihood the block size sets $k$ for all trailing updates, and the predicted choice changes with both $k$ and the cap: with $k = 2048$, emulation at 1000 W (measured +21.7 against +4.1%) and truncation at 650 and 500 W (+5.5 and +8.3 against −9.7 and −19.8%), as predicted; with $k = 1024$ truncation everywhere, with $k = 4096$ emulation at 1000 and 650 W. The one miss is $k = 4096$ at 500 W, where emulation measures +10.8 against +7.5% (3.1% lost).

## VIII. Discussion

**Where truncation pays.** Three conditions decide the gain: the GPU must be power-bound, the truncated operands must flow directly and in volume into multipliers or data paths, and the algorithm must tolerate the error, ideally by correcting it. The model makes the first quantitative; the second explains why GEMM-dominated solvers gain 4–20% while codes whose intermediates regrow full mantissas (N-body, MFEM, SW4, LAMMPS) gain little; the third separates HPC solvers from AI weights. On GH200 the gain at the default cap is mainly energy; on MI350X, whose default limit binds for FP64, BF16 and FP8 matrix work, truncation turns into 4.6–10% more speed at stock settings. A part with a higher limit or liquid cooling that is not power-bound at stock settings would see energy instead of speed, in an amount the model predicts from $P - P_0$. Truncation is not an alternative to clock tuning: at a fixed clock it lowers the energy per operation (by 8–38% in Table III), so it lowers the cost of whatever operating point a cap or a frequency choice selects. As throughput grows faster than power envelopes, running at the limit becomes the default operating point, and the energy spent toggling unneeded bits becomes a first-order performance parameter.

**Truncation and emulation.** Emulation is the stronger lever where it can amortize its fixed costs: long inner dimensions, generous caps, narrow exponent spans, ample memory and few distinct shapes. Truncation is the better choice elsewhere, and of the two strategies compared here the only one that applies outside matrix multiplication and to BF16 and FP8; in HPL the measured order at 500 W reverses only when the residual threshold is tightened below the benchmark's own. The selector of Section VII-C makes this choice per call from GEMM measurements alone; building it into a BLAS dispatcher, with a cheap span check before emulating, remains to be done.

**AI workloads.** Truncation does not carry over to model weights, which no residual corrects: truncating the frozen FP8 base weights of Llama 2 70B LoRA fine-tuning to 2 of 3 mantissa bits made steps 1% faster but needed 22–39% more samples to reach the target loss (GH200; Supplement S7).

**Benchmark rules.** TOP500 rules require full FP64 arithmetic [45], and HPL-MxP rules require at least 16-bit floating point and allow diagonally dominant matrices for testing only [42]; all HPL and HPL-MxP numbers here are research results, not official scores. Scientific software has no such restriction; the criterion is whether physical results stay within the required accuracy.

**Limitations.** (i) Two platforms, one generation each, and at most two nodes (16 GPUs) on MI350X; other parts manage power and cool differently. (ii) Safe bit counts depend on platform and problem size and must be validated (HPL 27 bits: passes on GH200 at three of four sizes, fails on MI350X at all). (iii) HPL-MxP results hold for a diagonally dominant test matrix only. (iv) Each scientific code was tested on one or two inputs; on MI350X DF-MP2 uses our own contraction of PySCF's tensor. (v) Analysis units are runs, windows or rounds within one or two nodes and may be correlated within a job; GH200 intervals treat pairs within a node set as independent, and node sets differed more than pairs within a set. (vi) Temperature confounds power-capped measurements; ABBA cancels linear drift but not fan feedback, so we report temperature-adjusted values beside raw ones and checked them under controlled thermal history for GEMM and HPL-MxP only; fan speed could not be fixed. (vii) Default caps are also the highest, not uncapped states, and the limiters' time scales are unknown. (viii) Power and energy come from the devices' own sensors and accumulators, not an external meter; GH200 module power includes the Grace CPU. (ix) The MI350X clock reduction attributed to clock stretching is an inference. (x) About half of the Quantum ESPRESSO gain on the gold surface comes from fewer Davidson iterations, for unidentified reasons. (xi) The GEMMul8 memory limits and the size failure are observations about one library build.

## IX. Conclusion

On GPUs of two vendors, truncating unused mantissa bits in software buys performance for HPC applications under power limits while their results stay within each code's own criterion. The saving comes from bits that change between successive operands, and a first-order model predicts a truncated kernel's performance–cap curve from the native curve, the idle power and one locked-clock measurement. MI350X is power-bound at its stock limit, so truncation speeds up HPL, HPL-MxP on a diagonally dominant test matrix, a Gaussian-process likelihood and DF-MP2 by 4.6–10% at default settings and 7–19% at 500 W. It is one of two ways to spend less energy per useful FP64 operation and not the faster one everywhere: INT8 emulation leads for long inner dimensions at generous caps, truncation for short inner dimensions, tight caps and strict tolerances, and a selector fitted on GEMM measurements chooses between them per call almost without loss on a held-out application. Truncation also keeps the format, needed no extra memory in our in-program implementation and applies to BF16 and FP8. Safe bit counts must be validated on the target platform, and power-capped measurements must control temperature. All HPL and HPL-MxP numbers are research results, not official scores.

## References

[1] S. Bhalachandra, B. Austin, S. Williams, and N. J. Wright, "Understanding the impact of input entropy on FPU, CPU, and GPU power," arXiv preprint arXiv:2212.08805, 2022.

[2] T. Gregersen, P. Patel, and E. Choukse, "Input-dependent power usage in GPUs," in *SC24-W: Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 2024, pp. 1872–1877.

[3] O. Antepara *et al.*, "Benchmark-driven models for energy analysis and attribution of GPU-accelerated supercomputing," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '25)*, 2025, pp. 888–904.

[4] H. He, "Strangely, matrix multiplications on GPUs run faster when given "predictable" data! [short]," Thonk From First Principles (Substack blog), https://www.thonking.ai/p/strangely-matrix-multiplications, Apr. 2024.

[5] J. Y. F. Tong, D. Nagle, and R. A. Rutenbar, "Reducing power by optimizing the necessary precision/range of floating-point arithmetic," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 8, no. 3, pp. 273–286, 2000.

[6] J. Pool, A. Lastra, and M. Singh, "Precision selection for energy-efficient pixel shaders," in *ACM SIGGRAPH Symposium on High Performance Graphics (HPG)*, 2011, pp. 159–168.

[7] D. Guenther, T. Henriksson, R. Leupers, and G. Ascheid, "Mantissa-masking for energy-efficient floating-point LTE uplink MIMO baseband processing," in *Design, Automation & Test in Europe (DATE)*, 2016, pp. 1028–1029.

[8] P. V. Argade and D. N. Peroni, "Variable precision computing system," US Patent 11,455,766 B2, Sep. 27, 2022.

[9] K. Ozaki, Y. Uchino, and T. Imamura, "Ozaki scheme II: A GEMM-oriented emulation of floating-point matrix multiplication using an integer modular technique," arXiv preprint arXiv:2504.08009, 2025.

[10] Y. Uchino *et al.*, "High-performance and power-efficient emulation of matrix multiplication using INT8 matrix engines," in *SC '25 Workshops*, 2025.

[11] O. Antepara, L. Oliker, and S. Williams, "Roofline analysis of tightly-coupled CPU-GPU superchips: A study on MI300A and GH200," in *SC '25 Workshops*, 2025, pp. 1205–1216.

[12] J. Lucas and B. Juurlink, "ALUPower: Data dependent power consumption in GPUs," in *IEEE 24th International Symposium on Modeling, Analysis and Simulation of Computer and Telecommunication Systems (MASCOTS)*, 2016, pp. 95–104.

[13] B. Rountree, D. H. Ahn, B. R. de Supinski, D. K. Lowenthal, and M. Schulz, "Beyond DVFS: A first look at performance under a hardware-enforced power bound," in *IEEE 26th International Parallel and Distributed Processing Symposium Workshops & PhD Forum (IPDPSW)*. IEEE, 2012, pp. 947–953.

[14] Y. Inadomi *et al.*, "Analyzing and mitigating the impact of manufacturing variability in power-constrained supercomputing," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '15)*. ACM, 2015, pp. 1–12, article 78.

[15] T. Patki, D. K. Lowenthal, B. Rountree, M. Schulz, and B. R. de Supinski, "Exploring hardware overprovisioning in power-constrained, high performance computing," in *Proceedings of the 27th International ACM Conference on Supercomputing (ICS '13)*. ACM, 2013, pp. 173–182.

[16] J. You, J.-W. Chung, and M. Chowdhury, "Zeus: Understanding and optimizing GPU energy consumption of DNN training," in *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*. USENIX Association, 2023, pp. 119–139. [Online]. Available: https://www.usenix.org/conference/nsdi23/presentation/you

[17] J. Pool, A. Lastra, and M. Singh, "Energy-precision tradeoffs in mobile graphics processing units," in *IEEE International Conference on Computer Design (ICCD)*, 2008, pp. 60–67.

[18] A. Angerd, E. Sintorn, and P. Stenström, "A framework for automated and controlled floating-point accuracy reduction in graphics applications on GPUs," *ACM Transactions on Architecture and Code Optimization*, vol. 14, no. 4, 2017, article 46.

[19] R. Jahadi and E. Atoofian, "Low-power register file for tensor cores," in *IEEE 15th International Green and Sustainable Computing Conference (IGSC)*, 2024, pp. 159–164.

[20] N. Chatterjee *et al.*, "Architecting an energy-efficient DRAM system for GPUs," in *IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 2017, pp. 73–84.

[21] M. O'Connor *et al.*, "Fine-grained DRAM: Energy-efficient DRAM for extreme bandwidth systems," in *IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2017, pp. 41–54.

[22] S. Ghose *et al.*, "What your DRAM power models are not telling you: Lessons from a detailed experimental study," *Proceedings of the ACM on Measurement and Analysis of Computing Systems*, vol. 2, no. 3, 2018.

[23] C. Rubio-González *et al.*, "Precimonious: Tuning assistant for floating-point precision," in *Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis (SC13)*, 2013.

[24] N. J. Higham and T. Mary, "Mixed precision algorithms in numerical linear algebra," *Acta Numerica*, vol. 31, pp. 347–414, 2022.

[25] T. Grützmacher, T. Cojean, G. Flegar, H. Anzt, and E. S. Quintana-Ortí, "Acceleration of PageRank with customized precision based on mantissa segmentation," *ACM Transactions on Parallel Computing*, vol. 7, no. 1, pp. 1–19, 2020.

[26] A. C. I. Malossi *et al.*, "The transprecision computing paradigm: Concept, design, and applications," in *Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 2018, pp. 1105–1110.

[27] A. Haidar, S. Tomov, J. Dongarra, and N. J. Higham, "Harnessing GPU tensor cores for fast FP16 arithmetic to speed up mixed-precision iterative refinement solvers," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC18)*, 2018, pp. 603–613.

[28] J. Dongarra and P. Luszczek, "HPL-MxP benchmark: Mixed-precision algorithms, iterative refinement, and scalable data generation," *The International Journal of High Performance Computing Applications*, vol. 40, no. 1, pp. 52–62, 2026, published online Sep. 25, 2025; arXiv:2509.19618.

[29] M. Klöwer, M. Razinger, J. J. Dominguez, P. D. Düben, and T. N. Palmer, "Compressing atmospheric data into its real information content," *Nature Computational Science*, vol. 1, no. 11, pp. 713–724, 2021.

[30] K. Ozaki, T. Ogita, S. Oishi, and S. M. Rump, "Error-free transformations of matrix multiplication by using fast routines of matrix multiplication and its applications," *Numerical Algorithms*, vol. 59, no. 1, pp. 95–118, 2012.

[31] H. Ootomo, K. Ozaki, and R. Yokota, "DGEMM on integer matrix multiplication unit," *The International Journal of High Performance Computing Applications*, vol. 38, no. 4, pp. 297–313, 2024.

[32] NVIDIA, "cuBLAS library documentation, section 1.5: Floating point emulation (v13.4; archived 13.2.1)," https://docs.nvidia.com/cuda/cublas/index.html#floating-point-emulation, accessed Oct. 5, 2026.

[33] A. Schwarz *et al.*, "Guaranteed DGEMM accuracy while using reduced precision tensor cores through extensions of the Ozaki scheme," in *Proceedings of the Supercomputing Asia and International Conference on High Performance Computing in Asia Pacific Region (SCA/HPCAsia)*, 2026, pp. 91–101.

[34] S. Abdulah, H. Ltaief, Y. Sun, M. G. Genton, and D. E. Keyes, "Geostatistical modeling and prediction using mixed precision tile Cholesky factorization," in *IEEE 26th International Conference on High Performance Computing, Data, and Analytics (HiPC)*, 2019, pp. 152–162.

[35] S. Abdulah *et al.*, "Accelerating geostatistical modeling and prediction with mixed-precision computations: A high-productivity approach with PaRSEC," *IEEE Transactions on Parallel and Distributed Systems*, vol. 33, no. 4, pp. 964–976, 2022.

[36] Q. Cao *et al.*, "Reshaping geostatistical modeling and prediction for extreme-scale environmental applications," in *SC22: International Conference for High Performance Computing, Networking, Storage and Analysis*, 2022, pp. 1–12.

[37] Q. Cao, S. Abdulah, H. Ltaief, M. G. Genton, D. Keyes, and G. Bosilca, "Reducing data motion and energy consumption of geospatial modeling applications using automated precision conversion," in *IEEE International Conference on Cluster Computing (CLUSTER)*, 2023, pp. 330–342.

[38] J. Ren, H. Ltaief, S. Abdulah, and D. E. Keyes, "Accelerating mixed-precision out-of-core Cholesky factorization with static task scheduling," in *ISC High Performance 2025 Research Paper Proceedings (40th International Conference)*, 2025, pp. 1–12.

[39] J. Woo, S. Kim, and W. Y. Kim, "Dynamic precision approach for accelerating large-scale eigenvalue solvers in electronic structure calculations on graphics processing units," *Journal of Chemical Theory and Computation*, vol. 19, no. 5, pp. 1457–1465, 2023.

[40] H. Huang, W. Shao, and J. Hammond, "Accelerating density fitting with adaptive-precision and 8-bit integer on AI accelerators," arXiv preprint arXiv:2601.08077, 2026.

[41] NVIDIA, "NVIDIA GH200 Grace Hopper superchip datasheet," Document 3773000, linked from https://www.nvidia.com/en-us/data-center/grace-hopper-superchip/, Mar. 2025, accessed Oct. 5, 2026.
[42] HPL-MxP, "HPL-MxP rules," https://hpl-mxp.org/rules.md, accessed Oct. 5, 2026.
[43] P. Giannozzi *et al.*, "Quantum ESPRESSO toward the exascale," *The Journal of Chemical Physics*, vol. 152, p. 154105, 2020.
[44] X. Wu *et al.*, "Enhancing GPU-acceleration in the Python-based simulations of chemistry frameworks," *WIREs Computational Molecular Science*, vol. 15, no. 2, p. e70008, 2025.
[45] TOP500.org, "The Linpack benchmark," https://www.top500.org/project/linpack/, accessed Oct. 5, 2026.